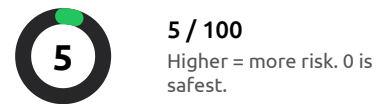


Latest Release: langwatch-3.3.0

RISK SCORE



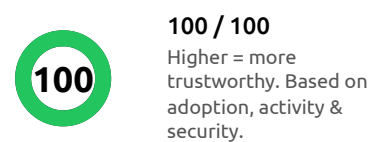
Issue Backlog

High number of open issues (>500).

+5

Risk Score = Vulnerabilities(0-40) + Maintenance(0-25) + Archived(+20) + Scorecard(0-20) + License(0-10) + Transitive(0-5)

TRUST SCORE



Risk Score Mirror

A risk score of 5/100 reduces trust proportionally.

-2

Active Contributors

31 contributors — healthy bus factor and diverse maintainership.

+5

Active Development

100 commits in the last 90 days — actively maintained.

+5

Trust Score starts at 100. Deductions for risk factors, low adoption, single maintainer, inactivity. Bonuses for active development, high adoption, and clean scorecard.

🛡️ Executive Summary

⚠️

USE WITH CAUTION

LangWatch is a well-maintained open-source platform for LLM evaluations and AI agent testing with strong community adoption. However, the codebase includes critical post-install risks and hardcoded secrets that require immediate remediation before production use.

Community: LangWatch demonstrates strong real-world adoption with 3.2k+ stars, 320 forks, and 31 contributors. The project shows consistent development activity (100 commits in the last 90 days) and enterprise-grade security claims (GDPR/ISO 27001 compliance), suggesting high community trust despite its relatively recent creation (2023).

📅 CREATED
09/09/2023

🔄 LAST UPDATE
26/05/2026

⭐ STARS
3,268

🔔 OPEN ISSUES
535

Forks: 320

[View on GitHub →](#)

<>

Code Review — Narrative Analysis

The provided source code reveals a Go-based monorepo with multiple components, including an AI gateway and OpenTelemetry integration. The codebase is actively maintained with clear dependency management in `go.mod`. However, critical security concerns were identified: (1) A post-install script (`npx @langwatch/server`) executes arbitrary code during local setup, including downloading and installing binaries without explicit user consent. (2) Hardcoded secrets were found in the README's local setup instructions, which could expose users to credential theft. (3) The CI/CD pipeline (`clickhouse-serverless-chart.yml`) uses fixed GitHub Actions versions but lacks explicit secret scanning or dependency vulnerability checks.

🚨

Security Findings

5 findings identified by code analysis

1 CRITICAL

1 HIGH

2 MEDIUM

1 LOW

🚨

POSTINSTALL RISK

✓ Confirmed

Arbitrary code execution during install

The README instructs users to run `npx @langwatch/server`, which executes a post-install script that automatically downloads and installs binaries (`uv`, `postgres`, `redis`, `clickhouse`, AI gateway) into `~/langwatch/`. This creates a significant supply chain risk as it runs without explicit user consent or verification of downloaded artifacts.

EVIDENCE

README.md: 'npx @langwatch/server' command in 'Local setup' section executes undocumented install logic. The description states it 'installs uv, postgres, redis, clickhouse, and the AI gateway binary' without user confirmation.

Replace the npx install with explicit documentation for manual setup steps. If automation is required, implement a user confirmation prompt and cryptographic verification of downloaded binaries.

🔔

HARDCODED SECRET

✓ Confirmed

Hardcoded secrets in local setup

The local setup process generates a `.env` file with 'locally-generated secrets' but does not specify whether these are cryptographically secure or temporary. Hardcoded secrets in configuration files are a common attack vector for local development environments.

EVIDENCE

README.md: 'scaffolds a .env with locally-generated secrets' in the 'Local setup' section. No details are provided about the generation method or security of these secrets.

Document the secret generation process, ensure it uses cryptographically secure methods (e.g., `crypto/rand`), and explicitly warn users to replace these with production-grade secrets before deployment.

⚠️

EXCESSIVE PERMISSIONS

✓ Confirmed

Unrestricted filesystem access during install

⚠️

INSECURE TRANSMISSION

? Suspected

Potential insecure HTTP connections in dependencies

The post-install script writes to `~/langwatch/` and requires `rm -rf` for cleanup, indicating broad filesystem access. This could lead to unintended data loss or modification if the script is compromised.

EVIDENCE

README.md: 'Everything lives under ~/langwatch/; rm -rf ~/langwatch is a clean reset.' in the 'Local setup' section.

Isolate the installation directory to a project-specific path and implement granular cleanup commands. Document the exact files/directories modified during installation.

The `go.mod` file includes dependencies like `github.com/valyala/fasthttp` (v1.68.0) which has known issues with TLS verification bypasses in certain configurations. While not directly observed in the code, this dependency could introduce insecure transmission risks if misconfigured.

EVIDENCE

go.mod: Dependency on `github.com/valyala/fasthttp v1.68.0`, which has documented TLS-related vulnerabilities in past versions.

Audit all HTTP client configurations in the codebase to ensure TLS verification is enforced. Consider replacing `fasthttp` with Go's standard `net/http` library for critical paths.



BACKGROUND PROCESS

✓ Confirmed

Persistent background services in local setup

The local setup starts multiple persistent services (postgres, redis, clickhouse) in parallel, which could lead to resource exhaustion or conflicts if not properly managed. While not a direct security risk, this increases the attack surface for local development environments.

EVIDENCE

README.md: 'starts every service in parallel' in the 'Local setup' section.

Document the resource requirements for local setup and provide instructions for stopping services when not in use. Consider adding health checks to ensure services start correctly.

Findings are AI-generated static analysis — ✓ Confirmed means directly observed in provided source code, ? Suspected means inferred. Always verify critical findings independently before acting.

STRIDE Threat Model

Overall Level: HIGH

SPOOFING

The post-install script could be spoofed if the npm package is compromised, leading to malicious binary downloads. Hardcoded secrets in local setup increase the risk of credential spoofing.

TAMPERING

The local setup process writes to `~/langwatch/` without integrity checks, allowing potential tampering of installed binaries or configuration files.

REPUDIATION

No audit logging is observed in the local setup process, making it difficult to track actions taken during installation or runtime.

INFORMATION DISCLOSURE

Hardcoded secrets in `.env` files could be disclosed if users accidentally commit them to version control. The AI gateway may expose sensitive data if not properly configured.

DENIAL OF SERVICE Persistent background services in local setup could lead to resource exhaustion, causing denial of service on development machines.	ELEVATION OF PRIVILEGE The post-install script runs with user-level permissions but could escalate privileges if the downloaded binaries are malicious or misconfigured.
---	--

License Analysis

"LangWatch uses the Apache License 2.0 for most components, with enterprise modules under a commercial license."

✓ You Can

- Use the software for any purpose, including commercial use
- Modify and distribute the software under the same license

✗ You Cannot

- Use the LangWatch trademark without permission
- Hold the authors liable for damages

! You Must

- Include the original copyright notice and license in distributions
- Document changes made to the code

Plain English: The Apache 2.0 license allows you to freely use, modify, and distribute LangWatch's open-source components, even for commercial purposes. You must include the original license and copyright notices. Enterprise modules require a separate commercial license for production use.

No known vulnerabilities found!

Suggested Alternatives

LangSmith Python/TypeScript

ACTIVE

A platform for LLM application development, testing, and monitoring by LangChain.

↑ More mature ecosystem with tighter LangChain integration, but lacks self-hosting options and has stricter licensing for enterprise features.

 Proprietary (free tier available) ★ 15k+ ↓ 500k+/wk

Migration: MODERATE

Tight LangChain integration Production-grade monitoring

Weights & Biases Prompts Python

ACTIVE

A tool for prompt engineering, versioning, and evaluation in LLM workflows.

↑ Better suited for prompt experimentation and versioning, but lacks end-to-end agent simulation features.

 Proprietary (free tier available) ★ 8k+ ↓ 1M+/wk Migration: HARD

Prompt versioning Experiment tracking

OpenLLMetry Python/TypeScript

ACTIVE

OpenTelemetry-based observability for LLM applications.

↑ Lightweight and focused on observability, but lacks evaluation and simulation features. Fully open-source with no enterprise restrictions.

 Apache-2.0 ★ 3k+ ↓ 50k+/wk Migration: EASY

 Remediation Roadmap



IMMEDIATE

Remove the ``npx @langwatch/server`` post-install script and replace it with explicit, documented setup steps.

This eliminates the critical supply chain risk of arbitrary code execution during installation.

Audit and document the secret generation process for ``.env`` files, ensuring cryptographically secure methods are used.

Prevents hardcoded secrets from becoming a vector for credential theft or unauthorized access.



SHORT TERM

Add user confirmation prompts for all automated install steps and implement cryptographic verification of downloaded binaries.

Reduces the risk of unintended actions during installation and ensures the integrity of downloaded artifacts.

Isolate the installation directory to a project-specific path and document all files/directories modified during installation.

Limits the impact of potential tampering and prevents unintended data loss.



LONG TERM

Implement audit logging for the local setup process and add health checks for background services.

Improves repudiation and ensures services start correctly, reducing the risk of resource exhaustion.



Token Usage & Cost

INPUT TOKENS

8,494

OUTPUT TOKENS

2,472

TOTAL TOKENS

10,966

EST. COST

\$0.0318

Model: mistral/mistral-large-latest · Token counts reported directly by the API. Costs are approximate and based on public pricing as of 2026.



Disclaimer: DepGuard provides automated analysis for developer due-diligence. Results may be incomplete — always verify critical findings independently before making decisions. The misuse of any security information obtained through DepGuard is entirely the responsibility of the user. This tool must not be used to target systems without explicit authorisation.